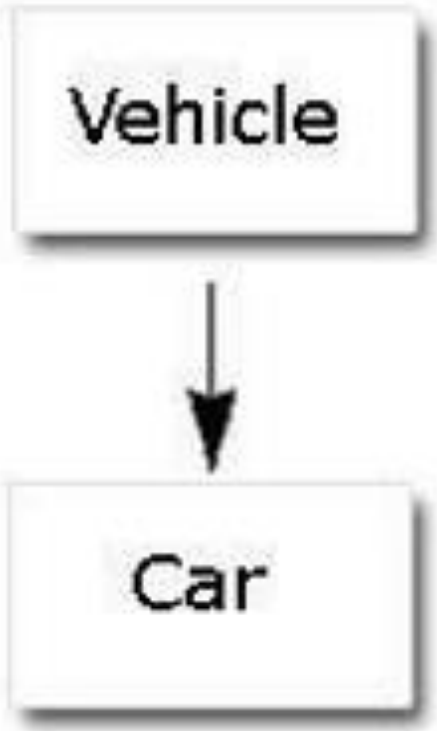
Inheritance is an OOP concept where one class (child/subclass) **acquires the properties and behaviors** (variables and methods) of another class (parent/superclass).

👉 It helps in **code reuse, faster development, and creating hierarchical relationships**.

Inheritance in Java is the mechanism where one class **inherits** the properties and methods of another class using the keyword **extends**.

- Parent class → **Super class / Base class**
- Child class → **Sub class / Derived class**

```
class Parent {
    // properties and methods
}
class Child extends Parent {
    // additional properties and methods
}
```



Simple Inheritance

Multilevel Inheritance

Single level/Simple Inheritance

When a subclass is derived simply from its parent class then this mechanism is known as simple inheritance. In case of simple inheritance there is only a sub class and its parent class. It is also called single inheritance or one level inheritance.

Example

```
class A
{
int x;       int y;
int get(int p, int q)
{
x=p; y=q;
return(0);
}
void Show()
{
System.out.println(x);
}
}
```

```
class B extends A
{
public static void main(String args[])
{

a.get(5,6);

}
 A a= new A();
a.Show();

void display()
{
System.out.println("y");          //inherited “y” from class A
}
}
```

Multilevel Inheritance

When a subclass is derived from a derived class then this mechanism is known as the multilevel inheritance.

The derived class is called the subclass or child class for it's parent class and this parent class works as the child class for it's just above (parent) class.

Multilevel inheritance can go up to any number of level.

```
class A
{
int x;
int y;
int get(int p, int q)
{
x=p; y=q;
return(0);
}
void Show()
{
System.out.println(x);
}
}
```

```
class B extends A
{
Void Showb()
{
System.out.println("B");
}
}
```

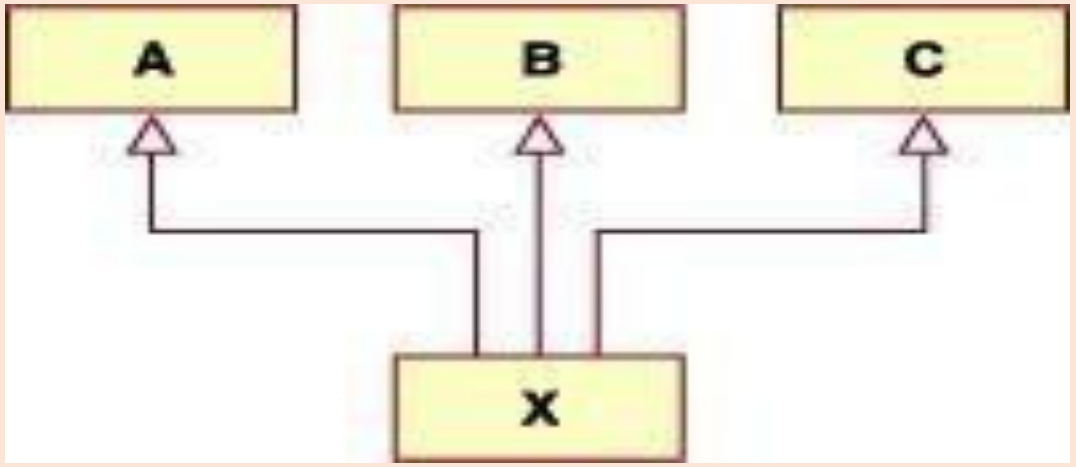
```
class C extends B
{
void display()
{
System.out.println("C");
}

public static void main(String args[])
{
A a = new A();
a.get(5,6);
a.Show();
}
}
```

OUTPUT 5

```
// Superclass
class Animal {
    void eat() {
        System.out.println("Animal is eating...");
    }
}

// Subclass
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking...");
    }
}

public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();   // inherited from superclass
        d.bark();  // subclass method
    }
}
```

Multiple Inheritance

The mechanism of inheriting the features of more than one base class into a single class is known as multiple inheritance.
Java does not support multiple inheritance but the multiple inheritance can be achieved by using the interface.



Super keyword

- The super is java keyword. As the name suggest super is used to access the members of the super class. It is used for two purposes in java.
- The first use of keyword super is to access the hidden data variables of the super class hidden by the sub class.
- Example: Suppose class A is the super class that has two instance variables as int a and float b. class B is the subclass that also contains its own data members named a and b. then we can access the super class (class A) variables a and b inside the subclass class B just by calling the following command. **super.member;**
- Use of super to call super class constructor: The second use of the keyword super in java is to call super class constructor in the subclass. This functionality can be achieved just by using the following command. **super(param-list);**

To access superclass variables

If subclass has a variable with the **same name** as the superclass, super helps to access the parent class variable.

```
class Animal {
   String color = "White";
}

class Dog extends Animal {
   String color = "Black";

   void printColor() {
      System.out.println("Dog color: " + color);
      System.out.println("Animal color: " + super.color);  // parent class
variable
   }
}
public class Test {
   public static void main(String[] args) {
      Dog d = new Dog();
      d.printColor();
   }}
```

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void eat() {
        System.out.println("Dog eats meat");
    }

    void show() {
        super.eat();  // calls parent method
        eat();        // calls subclass method
    }
}

public class Test {
    public static void main(String[] args) {
        new Dog().show();
    }}
```

To call superclass methods

If the subclass overrides a method, we can use super to call the **parent version** of the method.

Output:
Animal eats food
Dog eats meat

```
class Animal {
    Animal() {
        System.out.println("Animal constructor called");
    }}

class Dog extends Animal {
    Dog() {
        super();   // calls Animal constructor
        System.out.println("Dog constructor called");
    }}

public class Test {
    public static void main(String[] args) {
        new Dog();
    }}
```

To call superclass constructor

super() is used to call the parent class constructor.

Output:

Animal constructor called

Dog constructor called

Use of super	Purpose
super.variable	Access parent class variable
super.method()	Call parent class method
super()	Call parent class constructor

Method Overriding

Method Overriding occurs when a **subclass (child class)** provides its **own implementation** of a method that is **already defined in its superclass (parent class)** *with the same method name, same parameters, and same return type*.

Rules of Method Overriding

- name must be same.
- Parameters must be same.
- Return type must be same (or covariant).
- Method must be inherited.
- static, final, private methods cannot be overridden.

Why do we use Method Overriding?

- To **change** or **customize** behavior of a method inherited from parent.
- To provide **specific implementation** in child class.
- To achieve **runtime polymorphism**.
- To make code more **flexible and extensible**.

Polymorphism means **one name, many forms**.

In Java, polymorphism allows the same method name to behave **differently** depending on the object or context.

Simple meaning:

➡ **Same function behaves differently in different situations.**

Example from real life:

- A *person* can be a **teacher** at school,
- a **customer** at a shop,
- a **parent** at home.

Same person → different behavior.

Feature	Overloading	Overriding
Compile-time / Runtime	Compile-time	Runtime
Method Name	Same	Same
Parameters	Must be different	Must be same
Classes	Same class	Parent–child classes
Purpose	Add convenience	Change behavior

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }}

class Dog extends Animal {
    // Overriding the sound() method
    void sound() {
        System.out.println("Dog barks");
    }}

public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();  // Upcasting
        a.sound();             // Calls Dog's sound()
    }
}
```

Dog barks

```
class Parent {
    void display() {
        System.out.println("Parent display method");
    }}

class Child extends Parent {
    void display() {
        System.out.println("Child display method");
        super.display();  // calling parent method
    }}

public class Demo {
    public static void main(String[] args) {
        Child c = new Child();
        c.display();
    }}
```

Child display method
Parent display method

Abstract Class

- **abstract keyword** is used to make a class abstract.
- Abstract class can’t be instantiated with new operator.
- We can use abstract keyword to create an abstract method; an abstract method doesn’t have body.
- If classes have abstract methods, then the class also needs to be made abstract using abstract keyword, else it will not compile.
- Abstract classes are used to provide common method implementation to all the subclasses or to provide default implementation.

```
import java.util.Scanner;

abstract class Shape {
    abstract void area();   // abstract method
}

class Circle extends Shape {
    double r;
    Circle(double r) {
        this.r = r;
    }

    void area() {
        double a = 3.14 * r * r;
        System.out.println("Area of Circle = " + a);
    }
}
```

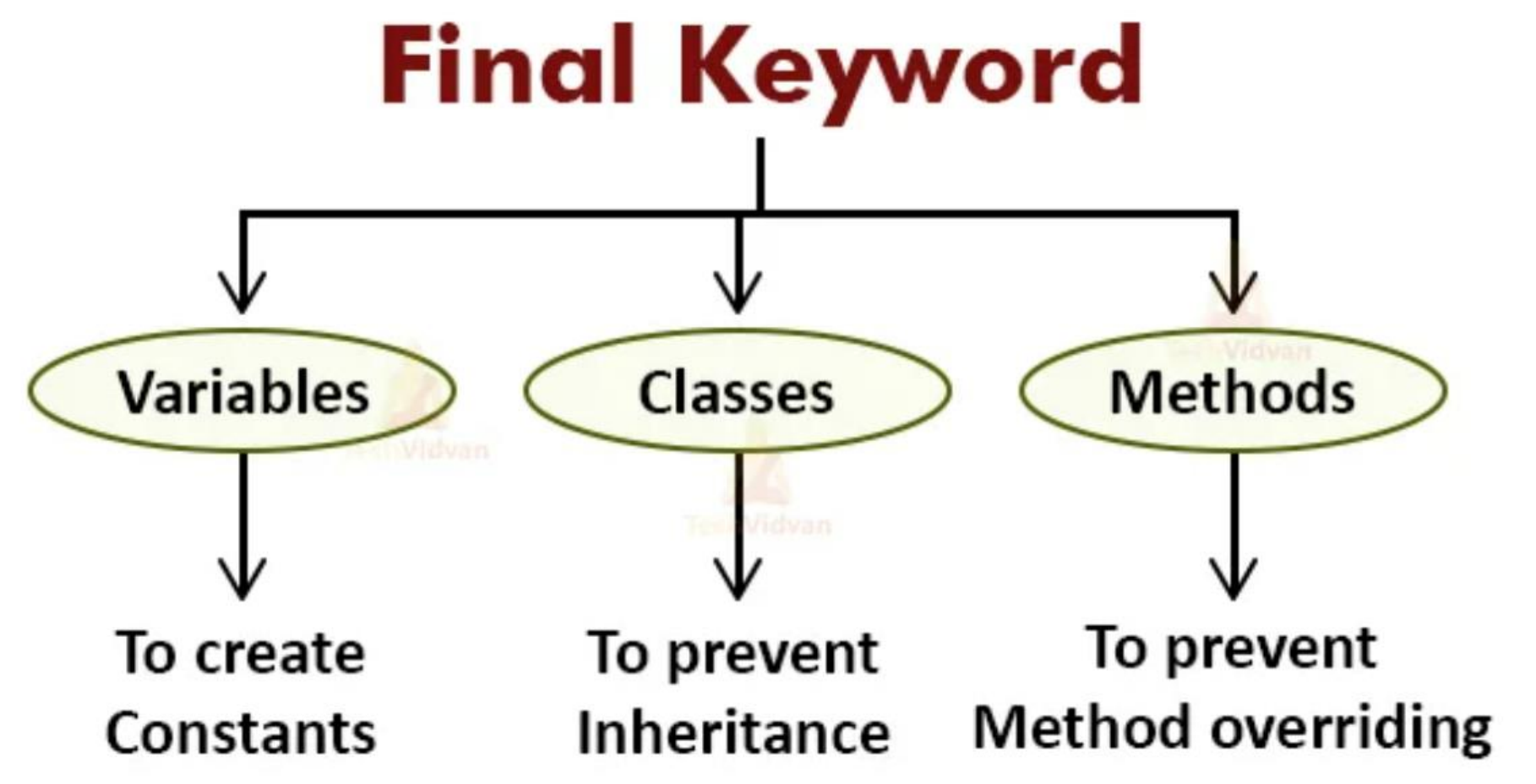
- Shape is an **abstract class**, so **you cannot create an object of Shape**.
- It has an **abstract method** area(), which has **no body**.
- Every child class **must** implement the area() method.

```
class Rectangle extends Shape {
    double l, b;

    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }

    void area() {
        double a = l * b;
        System.out.println("Area of Rectangle = " + a);
    }
}
```

```
class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

void area() {
        double a = 0.5 * base * height;
        System.out.println("Area of Triangle = " + a);
    }
}
```

```
public class TestShape {
    public static void main(String[] args) {

        Shape s;

        s = new Circle(5);
        s.area();

        s = new Rectangle(4, 6);
        s.area();

        s = new Triangle(10, 8);
        s.area();
    }
}
```

Using final with Inheritance



The final keyword in Java is used to restrict something.

final with Classes

If a class is declared as final, you cannot extend (inherit) that class.

```
final class A {
 void show() {
 System.out.println("A class");
  }}


 class B extends A { }  // ❌ Error: cannot inherit from final class
```

✔ final with Methods

If a method is final, child classes cannot override it.

```
class A {
    final void display() {
        System.out.println("Final method");
    }
}

class B extends A {
    // void display() { }  // ❌ Error: cannot override final method
}
```

final with Variables

A final variable is a constant – value cannot be changed.

```
final int x = 10;
x = 20;   // ❌ Error
```

final used on	Meaning	Effect on Inheritance
class	cannot be extended	❌ inheritance blocked
method	cannot be overridden	❌ method overriding blocked
variable	cannot be reassigned	N/A

Local Variable Type Inference (var) and Inheritance

The keyword var (Java 10+) lets the compiler infer the type automatically, but:

- ✔ var is resolved at compile time, not runtime
- ✔ var does not support polymorphism
- ✔ var is used only for local variables

Example showing var with inheritance:

```
class Animal {
    void sound() {
 System.out.println("Animal sound"); }
}

class Dog extends Animal {
    void sound() {
System.out.println("Dog barks"); }
}

public class Test {
    public static void main(String[] args) {
        var a = new Dog();   // type inferred as Dog, not Animal
        a.sound();           // Dog barks
    }
}
```

•var a = new Dog();
→ Compiler decides **a is Dog**, not Animal
•So you **cannot** write:
var a = new Animal();
a = new Dog(); // ❌ Error: type mismatch

var does not support parent reference → child object polymorphism(because type is fixed during compile time)

Animal a = new Dog(); // ✔ valid (runtime polymorphism)
var a = new Dog(); // type = Dog
a = new Animal(); // ❌ Error

Object Class in Java

Every class in Java **implicitly inherits** from the Object class.

class A extends Object { } // automatically added by Java

So, all classes have methods from object

Method	Purpose
toString()	returns string representation of object
equals()	compares two objects
hashCode()	returns hash value of object
clone()	creates copy of object
getClass()	returns runtime class
finalize()	cleanup before garbage collection
wait(), notify(), notifyAll()	thread communication

```
class A {
    int x = 10;
}

public class Test {
    public static void main(String[] args) {
        A obj = new A();
        System.out.println(obj.toString());
        System.out.println(obj.hashCode());
        System.out.println(obj.getClass());
    }
}
```